NORTHERN UNIVERSITY
OF BUSINESS & TECHNOLOGY KHULNA
Knowledge for an Enlightening Future

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

UI/UX Design Document

ON

AI Programming Tutor & Bug Explainer

Course Code: CSE 4104

Course Title: Software Development III

SUBMITTED TO

Name: Md. Riaz Mahmud

Assistant Professor

Department of CSE

Northern University of Business & Technology, Khulna.

SUBMITTED BY

Team Name	CSE4104 - 7B - T04
Section	7B
Project Title	AI Programming Tutor & Bug Explainer
Member Names	M.M. Asfy Or Rayhan Alisha Rahman Nadia Afrin Fatima Tanjim Rafa
Student IDs	11230121106 11230121112 11230121121 11230321556
GitHub Repository	https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer

Date of Submission: 22/06/2026

Team Information

Team Name	CSE4104 - 7B - T04
Section	7B
Project Title	AI Programming Tutor & Bug Explainer
Team Leader	M.M. Asfy Or Rayhan
Member Names	M.M. Asfy Or Rayhan Alisha Rahman Nadia Afrin Fatima Tanjim Rafa
Student IDs	11230121106 11230121112 11230121121 11230321556
GitHub Repository	https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer

Contents

1. **Project Information** 1

2. **UI/Wireframe Screenshots** 6

3. **User Flow Diagram**.....13

4. **Development Roadmap** 14

5. **Team Task Distribution** 18

6. **Final Technology Stack** 20

UI/UX Design Document

AI Programming Tutor & Bug Explainer

1. Project Information

1.1 Project Category

Artificial Intelligence (AI) Based Educational Web Application.

1.2 Project Overview

AI Programming Tutor & Bug Explainer is an intelligent web-based learning platform designed to assist beginner and intermediate programmers in understanding programming concepts, identifying coding mistakes, and improving their problem-solving skills. The system acts as a virtual programming tutor that provides personalized guidance, code explanations, error analysis, and learning support through an interactive interface. The platform aims to reduce the difficulties faced by students when learning programming by converting complex technical explanations into simple and understandable language. It also helps users analyze code errors, understand compiler messages, and receive suggestions for writing cleaner and more efficient code.

1.3 Problem Statement

Many students and novice programmers struggle to understand programming concepts and debugging techniques. Compiler errors and runtime exceptions are often difficult to interpret, leading to frustration and reduced learning efficiency. Traditional learning resources may not provide immediate feedback or personalized assistance. As a result, learners spend a significant amount of time searching for solutions instead of focusing on understanding the underlying concepts. This project addresses these challenges by providing an AI-powered tutor capable of explaining programming concepts, identifying bugs, translating technical errors into simple language, and recommending code improvements.

1.4 Objectives

To develop an AI-powered programming learning assistant that helps users understand programming concepts and debug code more effectively. More specifically, the project aims to provide simplified explanations of programming concepts, to identify and explain coding errors in user-submitted code, and to translate complex compiler and runtime error messages into understandable language. It also seeks to recommend code improvements and best programming practices, to offer personalized learning support based on user interactions, and to create an interactive and user-friendly educational platform that encourages continuous learning.

1.5 Key Features

The platform brings together a range of features that work side by side to support the learning process. It includes secure user registration and authentication, an AI-based programming tutor, and tools for bug detection and explanation, compiler error interpretation, and code improvement suggestions. Learners can also receive clear programming concept explanations, repository and project structure explanations, and analysis of documents and learning materials such as lecture notes and PDFs. To support continuous progress, the system maintains a personalized learning history and a progress tracking dashboard, and it offers voice-based explanation support so learners can listen to the explanations instead of only reading them. An administrative monitoring panel allows the platform to be managed and supervised, giving administrators a clear view of activity and usage across the system.

1.6 Target Users

The platform is intended for a wide range of learners. Its primary users are undergraduate programming students and beginner software developers, along with self-learning programmers and coding bootcamp participants who need immediate, easy-to-understand guidance while they practice. Beyond individual learners, it can also be used by academic institutions and training centers as a supporting tool for programming education.

1.7 Development Tools and Technologies

Frontend

- React.js
- Vite
- Tailwind CSS
- Monaco Code Editor

Backend

- Node.js
- Express.js

Database

- MongoDB

Authentication

- JSON Web Token (JWT)
- Bcrypt Password Hashing

Artificial Intelligence

- Large Language Model (LLM) Integration
- Natural Language Processing Techniques

1.8 System Architecture

The system follows a three-tier architecture:

1. **Presentation Layer (Frontend)**
    - User Interface
    - Dashboard
    - Code Editor
    - Learning Tools
2. **Application Layer (Backend)**
    - Business Logic
    - Authentication Services
    - AI Processing Services
    - API Management
3. **Data Layer**
    - User Information
    - Learning History
    - Activity Records
    - Performance Tracking Data

1.9 Expected Outcomes

The proposed system is expected to improve programming learning efficiency and reduce the time required for debugging. By providing immediate and personalized feedback, it should enhance students' understanding of programming concepts and help them learn more confidently. In the long run, the platform aims to support independent learning and steady skill development, allowing users to solve coding problems on their own.

1.10 Conclusion

AI Programming Tutor & Bug Explainer is an educational technology solution that combines artificial intelligence with programming education. By providing concept explanations, bug analysis, and personalized guidance, the platform supports learners in developing stronger programming skills and becoming more confident in solving coding problems independently.

2. UI/Wireframe Screenshots

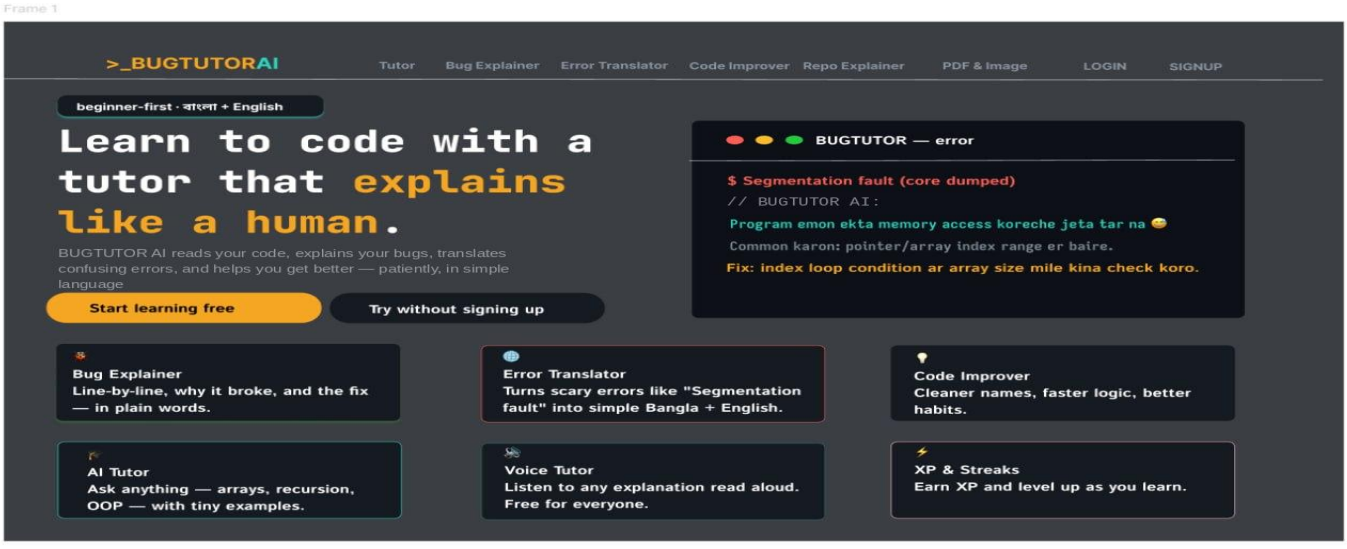
UI (User Interface) refers to how users interact with the system, including the pages, buttons, forms, dashboards and navigation. UX (User Experience) refers to how easy, efficient and enjoyable the system is to use.

A good UI/UX design should be:

- Simple and user-friendly
- Responsive across devices
- Consistent throughout the system
- Easy to navigate
- Focused on user needs

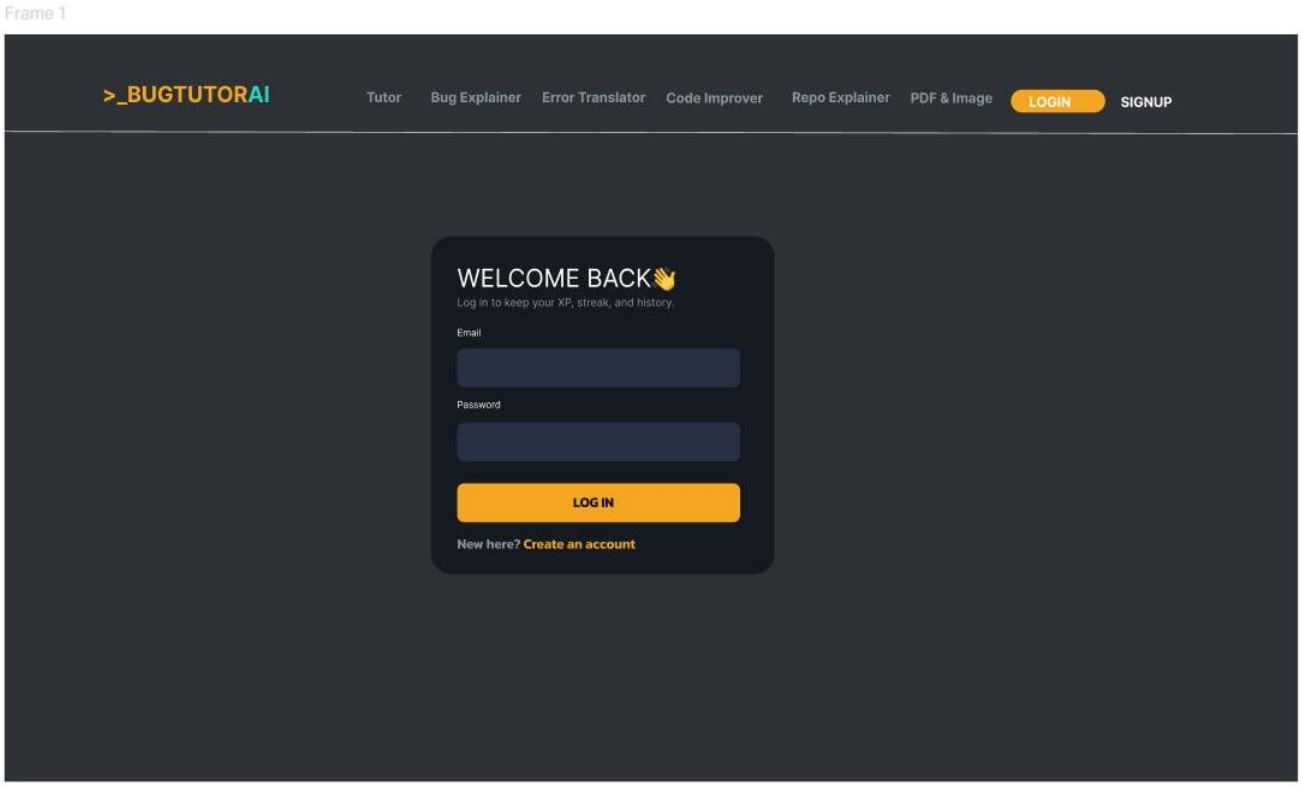
The screens below show the main interface of AI Programming Tutor & Bug Explainer. The design uses a dark theme with a single accent colour, a fixed top navigation bar and a consistent card layout, so the platform feels familiar from one page to the next. Each tool keeps the same simple pattern: an input area at the top and the AI explanation right below it.

2.1 Landing / Home Page



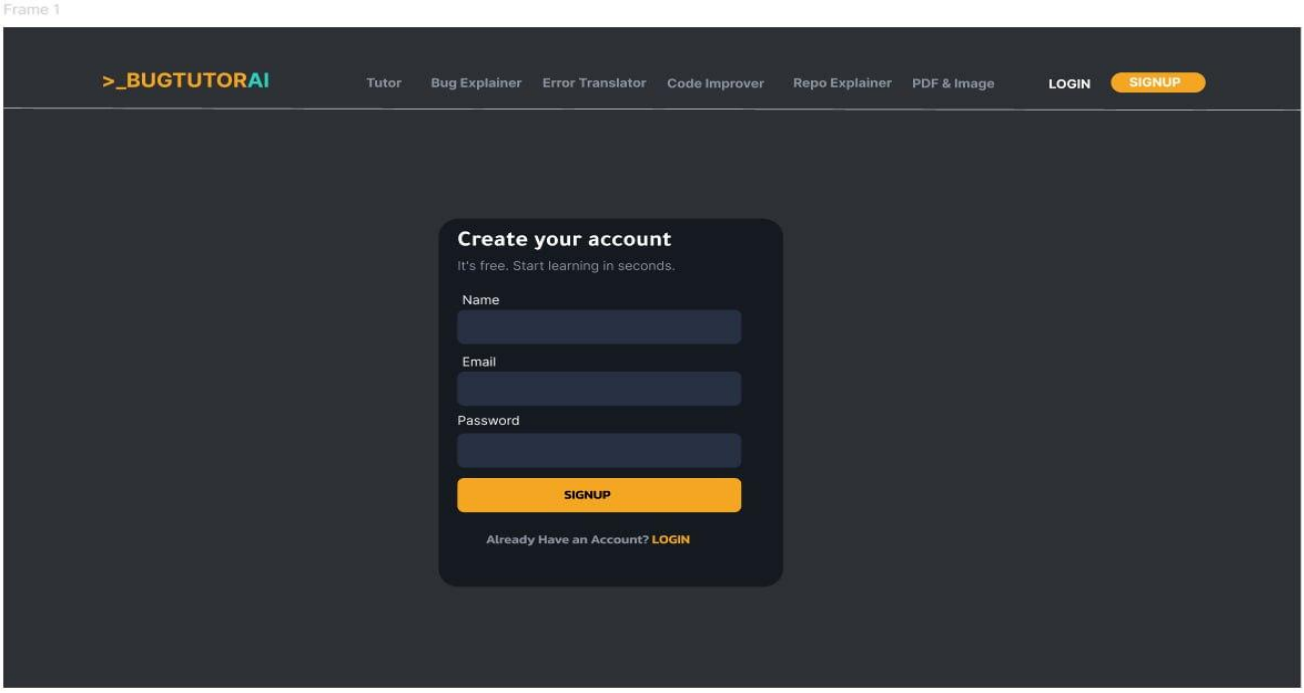
The entry page introduces the platform with the tagline "Learn to code with a tutor that explains like a human" and two clear actions: "Start learning free" and "Try without signing up". A sample error-explanation card on the right shows how the tutor replies, and the feature cards below highlight the Bug Explainer, Error Translator, Code Improver, AI Tutor, Voice Tutor and the XP & Streaks system. The top bar links to every tool along with Login and Signup.

2.2 Login Page



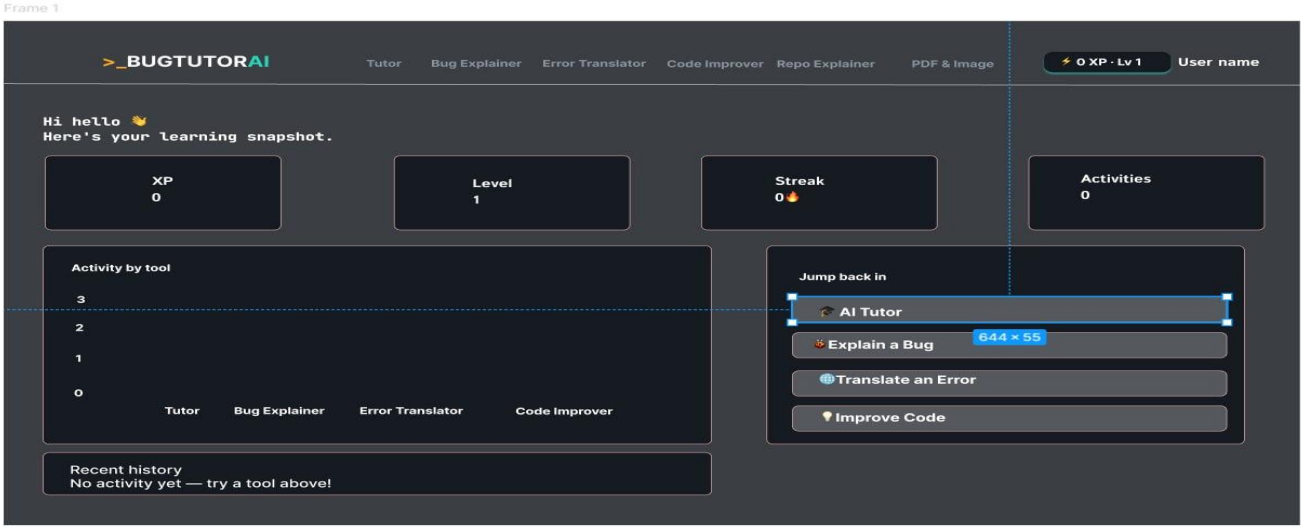
A simple "Welcome back" form with email and password fields and a single Log in button. A "Create an account" link sends new users to the signup page. Logging in keeps the user's XP, streak and history tied to their account.

2.3 Registration Page



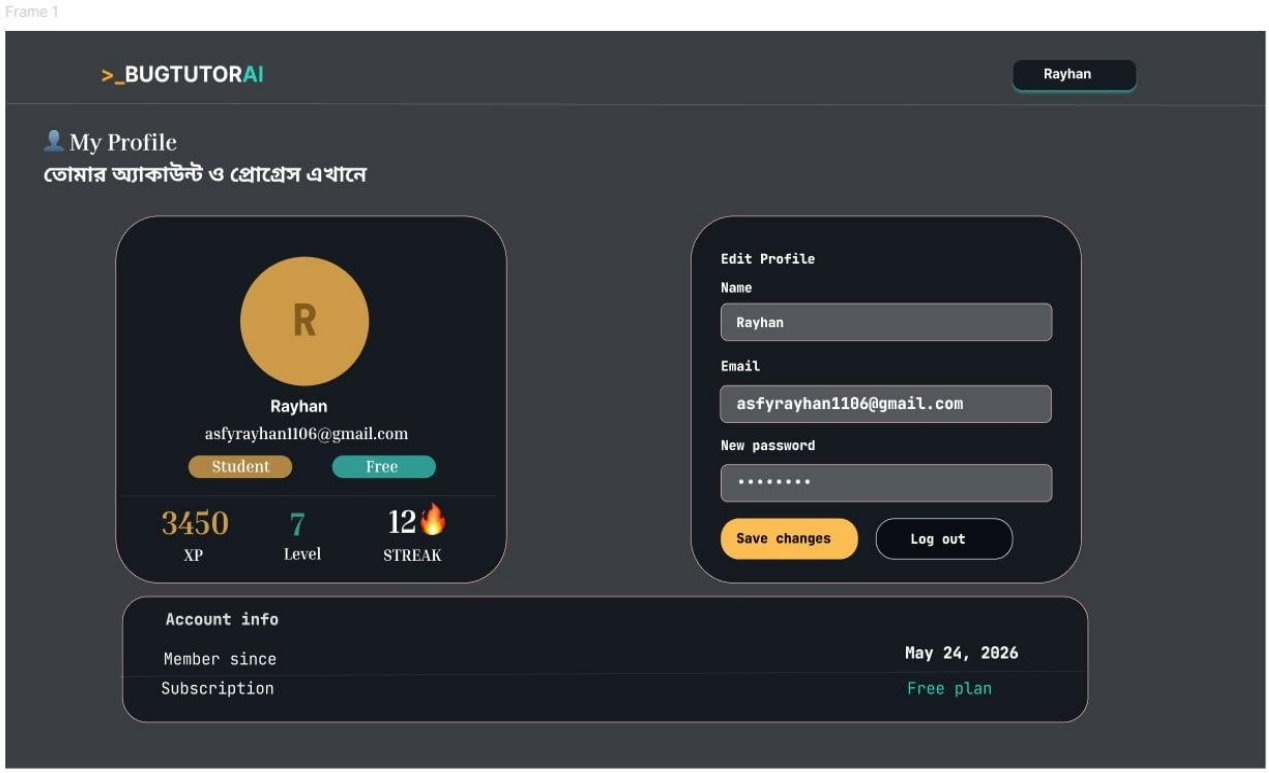
The "Create your account" form collects a name, email and password and signs the user up in a few seconds. Users who already have an account can switch to the login page using the link at the bottom.

2.4 Dashboard



The logged-in home greets the user and shows a learning snapshot. Stat cards display XP, Level, Streak and Activities, an "Activity by tool" bar chart shows how each tool has been used, and a "Jump back in" panel gives quick links to the AI Tutor, Bug Explainer, Error Translator and Code Improver. A recent history area sits at the bottom.

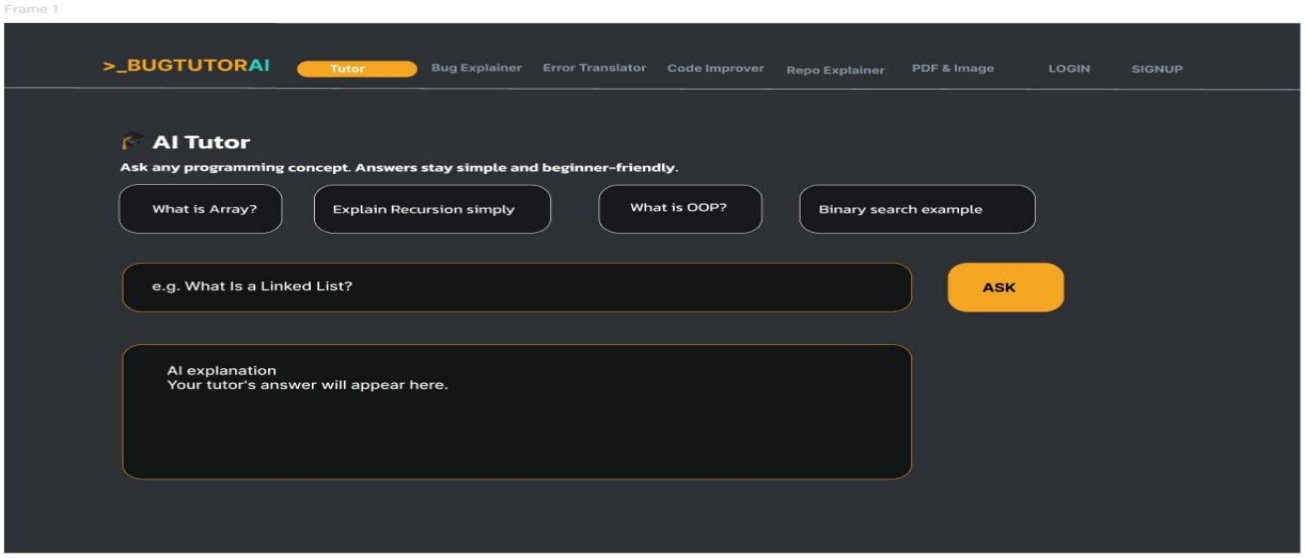
2.5 User Profile Page



The profile page shows the user's avatar, name and email, their plan badges (Student, Free) and their XP, Level and Streak. An Edit Profile panel lets them update their name, email and password with Save changes and Log out buttons, while an account info section lists the join date and subscription plan.

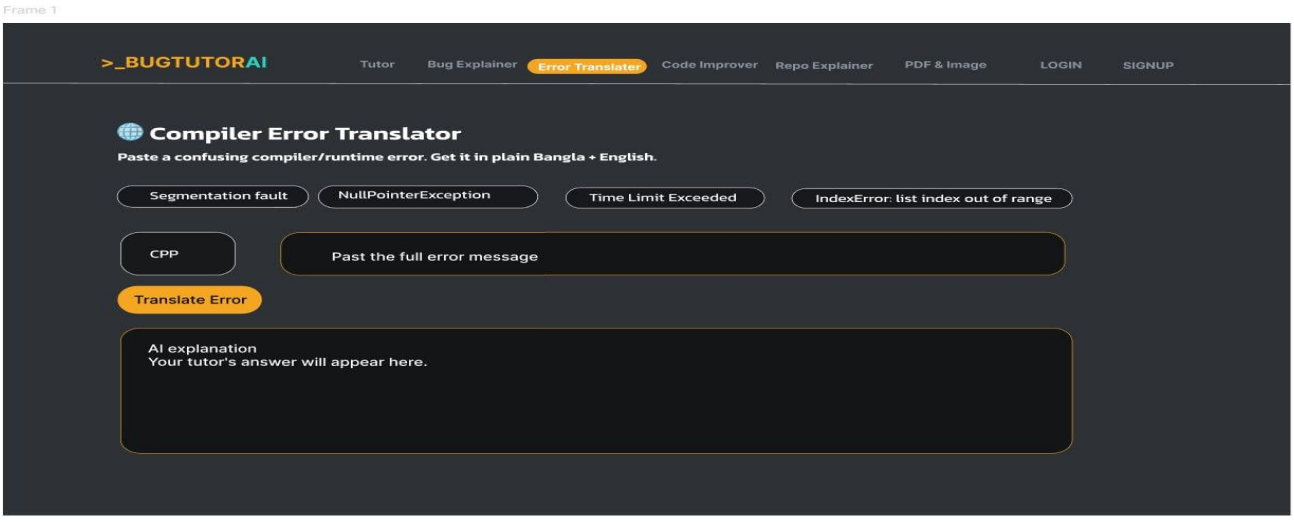
2.6 Main Functional Pages

2.6.1 AI Tutor Page



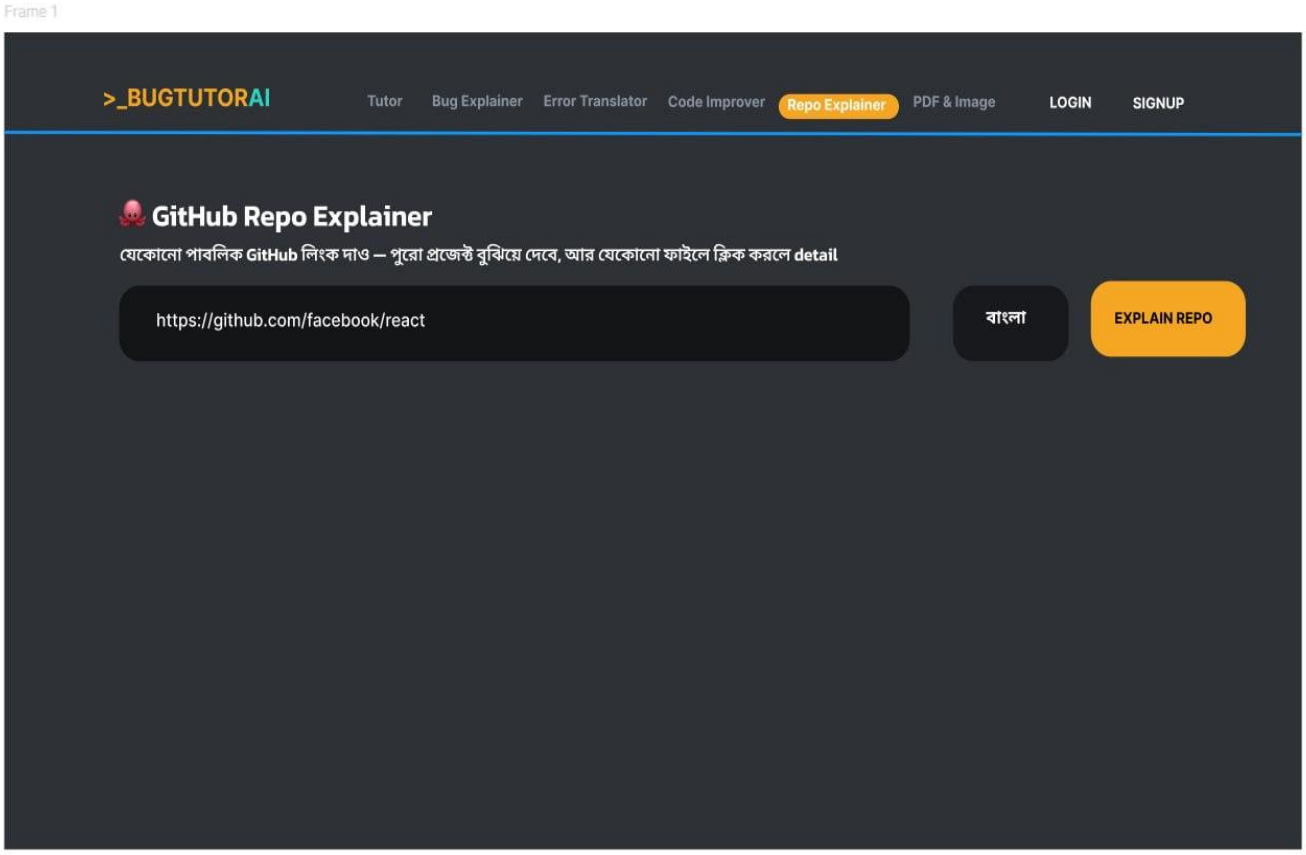
The AI Tutor lets a user ask about any programming concept. Quick-suggestion chips such as "What is Array?", "Explain Recursion simply" and "What is OOP?" make it easy to start, and the answer appears in the explanation area below the input box. This is one of the main AI interaction pages.

2.6.2 Compiler Error Translator Page



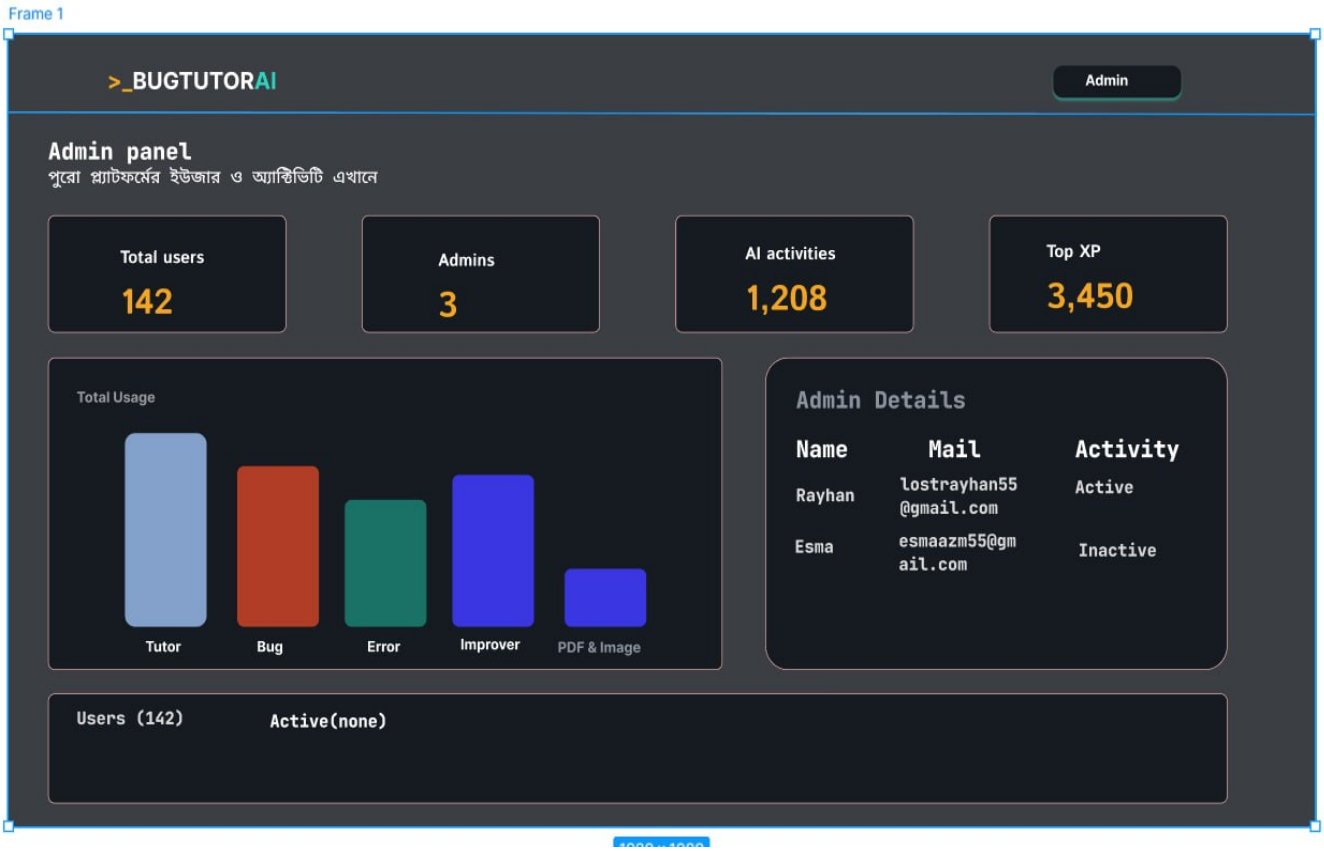
The flagship tool turns confusing errors into plain Bangla and English. Quick chips cover common errors like "Segmentation fault" and "NullPointerException", a language selector and a paste box let the user enter the full error, and the Translate Error button produces a beginner-friendly explanation in the answer area below.

2.6.3 GitHub Repo Explainer Page



This page takes any public GitHub repository link and explains the whole project in simple language. A language option (Bangla) and an Explain Repo button start the analysis, after which the user can click any file in the project to get a detailed explanation.

2.7 Admin Panel



The admin-only panel gives an overview of the whole platform. Stat cards show Total users, Admins, AI activities and Top XP, a Total Usage bar chart compares how much each tool is used, and an Admin Details table lists each admin's name, email and activity status alongside the full user list.

3. User Flow Diagram

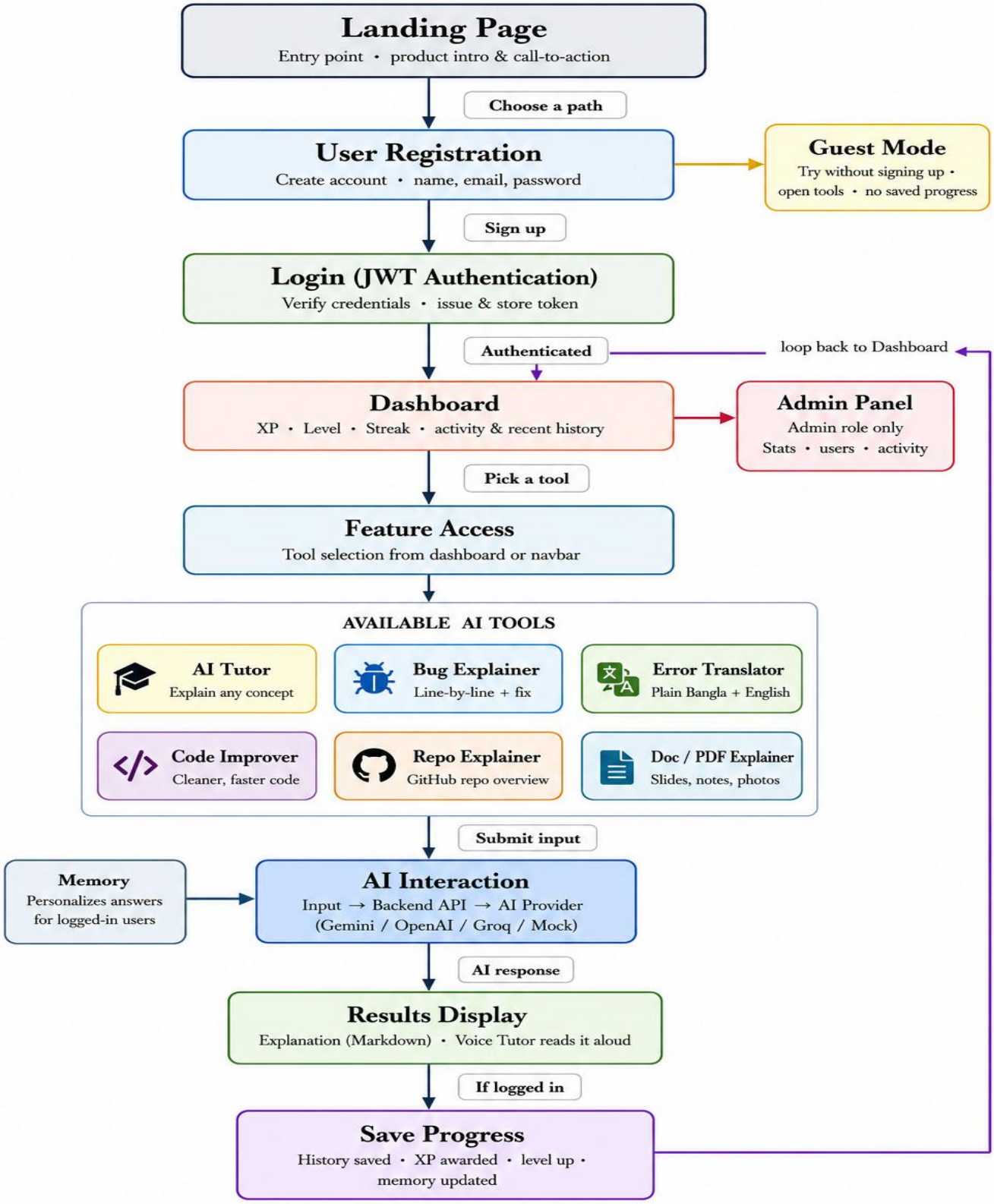


Figure: User Flow Diagram of the AI Programming Tutor and Bug Explainer

4. Development Roadmap

This document sets out our development plan for the rest of the semester. It lists the main modules of the system, the order we plan to build them in, how the work is divided among the team, and the schedule we expect to finish by. The plan runs from Week 06 to Week 12, with the last two weeks kept mostly for testing and deployment so we are not rushing those at the end.

4.1 Major Modules

The application is built around the following modules. Most of them are separate AI tools that share the same backend and login system.

Module	What it does
Authentication & User Management	Register and login with hashed passwords and JWT tokens, plus student/admin roles and protected pages.
AI Tutor	Explains a programming concept in simple language with a short analogy and a small example.
Bug Explainer	Takes pasted code and explains the bug line by line, then shows the fix.
Compiler Error Translator	Turns confusing errors like "Segmentation fault" into plain Bangla and English. This is our main feature.
Code Improver	Suggests better names, cleaner syntax and lower time complexity for a piece of code.
GitHub Repo Explainer	Reads a public GitHub repo URL and gives a plain overview plus a file tree where each file can be explained on demand.

Document Explainer (PDF & Image)	Accepts a lecture PDF, notes, or a photo of code and explains it for beginners.
Voice Tutor	Reads any explanation out loud using the browser speech engine. Free for everyone.
Memory & Personalization	Remembers what a logged-in user has worked on and which topics they struggle with, then personalizes answers.
Dashboard	Shows XP, level, streak, an activity chart and recent history.
Admin Panel	Lets an admin see platform stats, tool usage, the user list and recent activity, and promote or demote users.

4.2 Development Sequence

We are building the project bottom-up so that nothing is blocked waiting on something else. First comes the backend foundation: the Express server, the MongoDB connection and the login system. Once accounts and protected routes work, we add the AI service layer, which is a single place that talks to the AI provider so the rest of the code does not need to know which provider is used. After that we build the React frontend and connect the four core tools to the API. The heavier features (repo explainer, document explainer, voice, memory) come next because they depend on both the backend and the basic UI being in place. The dashboard, gamification and admin panel are added once there is real activity to display. The final stretch is testing and deployment. In short: backend and auth, then the AI layer, then the frontend tools, then the extra features, then dashboard and admin, and finally testing and release.

4.3 Team Responsibilities

The team is split into six areas. Members help across areas during the busy weeks, but each person owns one part so nothing falls through the cracks.

Role	Responsibilities
Frontend Development	React pages design, routing setup, code editor integration, charts/visualization components, overall UI/UX styling and look-and-feel
Backend Development	Express API routes, authentication & authorization logic, rate limiting middleware, history & XP calculation logic, admin endpoints
Database Management	MongoDB schema/model design, data validation rules, indexing & query optimization, data integrity & backup strategy
AI Integration	AI service layer (provider API calls), prompt design & engineering, provider/model setup & configuration, repo explainer & document explainer features, memory-based personalization, token usage/cost monitoring
Documentation	README & setup guide, API documentation, system architecture notes, final project report
Testing	Test plan creation, unit & integration testing, bug tracking, deployment to Render/Railway (backend) and Vercel (frontend)

4.4 Expected Completion Time

We expect a working build by the end of Week 10 and a fully tested, deployed version by the end of Week 12. The milestones below are how we will track progress.

Milestone	Target
Backend, database and login working	End of Week 06
Four core AI tools working through the API	End of Week 07
Frontend connected to all core tools	End of Week 08
All extended features added	End of Week 09
Dashboard and admin panel complete (feature complete)	End of Week 10
Testing finished and major bugs fixed	End of Week 11
Deployed, documented and ready to present	End of Week 12

5. Team Task Distribution

This document shows how the work on the AI Programming Tutor & Bug Explainer is divided among our team. Each member owns a main area, but we help each other during the heavier weeks so the whole project stays on track.

Team Members

Name	Student ID	Responsibility
Nadia Afrin	11230121121	Frontend Development
Fatima Tanjim Rafa	11230321556	Backend Development
Alisha Rahman	11230121112	Database Management
M.M. Asfy Or Rayhan	11230121106	AI Integration, Documentation, Testing

Task Distribution in Detail

The table below breaks down what each area covers in this project.

Task Area	Assigned Member	Main responsibilities
Frontend Development	Nadia Afrin 11230121121	Build all React pages (landing, login, register, dashboard and the AI tool pages), set up routing and the login state, integrate the Monaco code editor, the navbar, result cards and the voice button, and connect the interface to the

		backend with Axios. Also handles Tailwind styling and the dashboard charts.
Backend Development	Fatima Tanjim Rafa 11230321556	Build the Express server and all API routes, the authentication system (register/login with JWT and bcrypt), the middleware, rate limiting and error handling, and the controllers for the AI tools, repo, document and admin endpoints, including the XP and history logic.
Database Management	Alisha Rahman 11230121112	Set up the MongoDB Atlas cluster and the Mongoose connection, design the data models (User, ChatHistory, WeakTopic and Memory), and handle the queries that feed the dashboard, history and admin panel.
AI Integration	M.M. Asfy Or Rayhan 11230121106	Build the AI service layer that switches between providers (Groq, Gemini, OpenAI and the mock fallback), write the prompt templates for each tool, and implement the repo explainer (GitHub API), the document explainer (PDF and image) and the memory-based personalization.
Documentation	M.M. Asfy Or Rayhan 11230121106	Prepare the project documents: the README and setup guide, the API reference, the user flow, roadmap and technology documents, and the final report.
Testing	M.M. Asfy Or Rayhan 11230121106	Test every tool from start to finish, check the login and rate-limit behaviour, confirm the project runs in mock mode with no API key, review the interface on different screen sizes, and keep track of bugs until they are fixed.

6. Final Technology Stack

This document confirms the final technology stack for the AI Programming Tutor & Bug Explainer. Everything listed here is what the project actually uses. The stack was chosen to stay beginner-friendly and to run on free tiers, so the whole project can be developed and demonstrated at almost no cost.

Fronted

Technology	Purpose
React 18	Main UI library for building the pages and components.
Vite 5	Build tool and development server (fast reloads).
Tailwind CSS 3	Utility-first styling for a consistent look.
React Router 6	Page routing and protected routes.
Monaco Editor	The in-browser code editor used in the tools (same engine as VS Code).
Axios	HTTP client for calling the backend API.
Framer Motion	Small animations on the landing and tool pages.
Recharts	Charts on the dashboard and admin panel.
Web Speech API (browser)	Text-to-speech for the Voice Tutor. Built into the browser, so it is free.

Backend

Technology	Purpose
Node.js	JavaScript runtime for the server.
Express 4	Web framework that handles the API routes.
Mongoose 8	Models and queries for MongoDB.
JSON Web Token	Login tokens for authentication.
bcryptjs	Hashing passwords before storing them.
Multer	Handling PDF and image uploads in the document explainer.
unpdf	Reading text out of uploaded PDFs.
express-rate-limit	Limits requests on the AI routes to protect the API quota.
CORS & dotenv	Cross-origin access and environment configuration.

Database

Technology	Purpose
MongoDB (Atlas)	Stores users, chat history, weak topics and memory notes. We use the free Atlas cluster.

AI Services

The backend talks to the AI provider through one service file, so the provider can be switched from a single setting. The mock provider means the project still runs with no API key at all.

Service	Purpose
Groq (Llama 3.3 70B)	Primary provider. Free tier and OpenAI-compatible, so it costs nothing to run.
Google Gemini (1.5 Flash)	Alternative provider with a free tier.
OpenAI (gpt-4o-mini)	Optional paid provider for higher-quality answers.
Mock provider	Built-in offline fallback used for demos and development with no key.

External APIs

Service	Purpose
GitHub REST API	Reads public repositories for the Repo Explainer. No token needed.

Deployment

Platform	Used for
Render or Railway	Hosting the Node/Express backend.

Vercel	Hosting the React frontend.

Development Tools

Tool	Used for
Nodemon	Auto-restarting the server while developing.
Git & GitHub	Version control and team collaboration.

System Integration and Data Flow

The React frontend never calls the AI provider directly. It sends requests to the Express API, which handles login, talks to MongoDB and forwards prompts through the AI service to the chosen provider. Keeping the API keys on the server means they stay private, and swapping providers only takes one change in the environment file.

Data flow: React (browser) → Express API → MongoDB, with the API also reaching out to the AI provider and the GitHub API when needed.